

WaterX

Audit Report

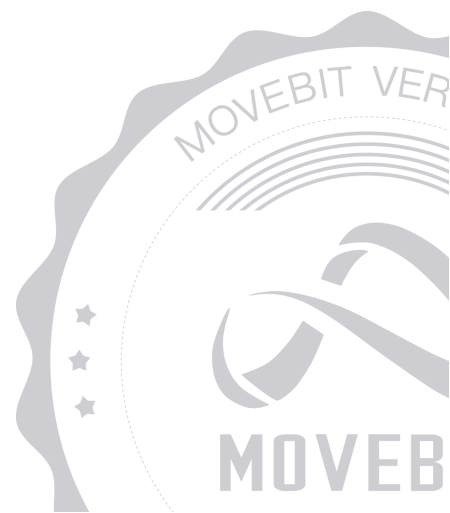


contact@bitslab.xyz



https://twitter.com/movebit_

Sun Apr 19 2026



WaterX Audit Report

1 Executive Summary

1.1 Project Information

Description	WaterX Perp is a perpetual futures decentralized exchange (DEX) on the Sui blockchain that utilizes a shared liquidity pool (WLP) as the universal counterparty for all trades. The protocol features a robust architecture including cross-margin account abstraction and a "hot-potato" transaction pattern to ensure secure and efficient execution of orders.
Type	DeFi
Auditors	Fishmen,Nebula,Sprig
Timeline	Thu Apr 02 2026 - Sun Apr 19 2026
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/WaterXProtocol/waterx-sdk-contract https://github.com/WaterXProtocol/waterx-contract
Commits	https://github.com/WaterXProtocol/waterx-sdk-contract:cd6455fbe2ceef9f6105fb6229fdcefc9b4711b7 https://github.com/WaterXProtocol/waterx-contract:cbffa8e044b00dffaaf53569fe9e72ca8abe36a

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
ACC	contracts/bucket_framework/sources/account.move	25ed2d116bfb571e3b74fbc14add66f47804157c
FLO	contracts/bucket_framework/sources/float.move	59511d2ba6e022dad045e22f80c734a9274a7a97
SHE	contracts/bucket_framework/sources/sheet.move	f4f49921ae43df8aa297c75600a349f682ed4111
LIA	contracts/bucket_framework/sources/liability.move	ffc2ec404c6a9f47a1646ef45ca31faa44315269
DOU	contracts/bucket_framework/sources/double.move	eebe06e7fc72adc7540c322b956c69441ad742f2
LTA	contracts/bucket_framework/sources/linked_table.move	cea86dca4d3e9b9334a4288ea15b1b98c57ea069
MUS	contracts/mock_usdc/sources/mock_usdc.move	a9d0a5b050a5c4edb440b62cdb1c37aaff6af9a
EVE	contracts/waterx_perp/sources/events.move	ab903fac33921b956a10e92feda8e0628d015ba9
WIT	contracts/waterx_perp/sources/witness.move	24fa2029ef6500660351d5020c72460d0cc37087
POS	contracts/waterx_perp/sources/position.move	e6d0382449476539d12bfd7199285fe27d62513f

ADM	contracts/waterx_perp/sources/ad min.move	9f73646bc9d43d09fc8787208baba 257e6f3530e
MAT	contracts/waterx_perp/sources/ma th.move	ea157c61b6f2322ff5439d1b2a8d2 8e633f331b3
GCO	contracts/waterx_perp/sources/glo bal_config.move	7fb2d890d1619c27123747b65d2c d46caca12169
RES	contracts/waterx_perp/sources/res ponse.move	0885482faf88b1f76b77ef506bc94c 11e8eca924
VIE	contracts/waterx_perp/sources/vie w.move	9431d0f64696e62fbdaf58c58b911 a183e981ffe
MCO	contracts/waterx_perp/sources/ma rket_config.move	9651ee7e9c8d8f37a858ca5110790 46e34bd2a8e
UAC	contracts/waterx_perp/sources/us er_account.move	caba9d069925b4b5ed0a7c74a3e7 eca1815af4c9
ERR	contracts/waterx_perp/sources/err or.move	d5d9222f5b84f4d5dc20492089080 205af7bb619
VER	contracts/waterx_perp/sources/ver sion.move	59307eebdb04f665ba7ee05bc1bb af0039b39554
REQ	contracts/waterx_perp/sources/re quest.move	2486548d9a1565ca763ce02d90fc4 27826b62000
RTA	contracts/waterx_perp/sources/ref erral_table.move	a94b39870631647c2956bc00ffd65 802dab129bd
TRA	contracts/waterx_perp/sources/tra ding.move	c62ade27d0827154c6f68002409a7 386caba3ece

LPO	contracts/waterx_perp/sources/lp_pool.move	e8020798522b24b08110c5d1f5b37394d9622a96
KBV	contracts/waterx_perp/sources/keyed_big_vector.move	66d51e0b1307382916865c750090accfac2e3c99
WBT	contracts/waterx_coins/sources/waterx_btc.move	8e7af2275b637f4b3b4a4696d4facf7b2328ca4c
WET	contracts/waterx_coins/sources/waterx_eth.move	e19ed07d5f94844643881ac3144cf8e3defede1c
COL	contracts/bucket_oracle/sources/collector.move	7116cbe606725e7f5cdac6a472ff97aa6a06a0ad
LIS	contracts/bucket_oracle/sources/listing.move	2e0c19ec073aeb0548532377af6890585a619749
RES1	contracts/bucket_oracle/sources/result.move	68a2fb242894d66bfceddb35a2f4bcb482a3f51e
AGG	contracts/bucket_oracle/sources/aggregator.move	ecfae1fd6f7ee1de2ec4b1950013dcc3f55bf38f
WLP	contracts/wlp/sources/wlp.move	adee37dc6b094dbd0d3746d8db3babd971d91830
MUS1	contracts/mock_usdt/sources/mock_usdt.move	16c6cdd54a59eecf785d7473217e249c2656ed49
PRU	contracts/pyth_rule/sources/pyth_rule.move	a82abcf2913f6ec9c516f76bc9bddbf608f35c0d

1.3 Issue Statistic

Item	Count	Fixed	Partially Fixed	Acknowledged
Total	17	6	1	10
Critical	2	2	0	0
Major	2	2	0	0
Medium	1	1	0	0
Minor	7	1	1	5
Informational	5	0	0	5

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by [waterx](#) to identify any potential issues and vulnerabilities in the source code of the [WaterX](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 17 issues of varying severity, listed below.

ID	Title	Severity	Status
FLO-6	<code>pow()</code> Function May Abort Unnecessarily	Informational	Acknowledged
LIS-3	Potential Performance Issue from Using <code>vec_map</code> in <code>aggregator_map</code>	Minor	Acknowledged
LPO-9	Missing Interval Catch-Up in <code>lp_pool::update_borrow_rates</code> Causes Permanent Borrow Fee Undercharge	Major	Fixed
PRU-1	Simplify Price Freshness Check by Using <code>get_price_no_older_than()</code>	Informational	Acknowledged
PRU-2	Incomplete Handling of expo Sign in <code>pyth_rule.feed()</code>	Minor	Acknowledged
SHE-5	<code>try_into_float()</code> Contains Typo in Comment	Informational	Acknowledged
WLP-4	Dependence on Centralized Storage Service for image URL	Informational	Acknowledged

GCO-13	Missing Events in Admin Functions	Minor	Acknowledged
KBV-19	State Bloat Caused by Dynamic Fields Leaking in <code>drop</code> of <code>KeyedBigVector</code>	Minor	Acknowledged
LPO-20	Inconsistent Usage of Basis Point Scale Constant May Lead to Maintainability Issues	Informational	Acknowledged
MCO-10	Missing Multi-Interval Funding Accrual in <code>market_config::update_funding_rate</code>	Major	Fixed
POS-21	Incorrect Borrow Fee Calculation Based on Collateral Amount in <code>calculate_borrow_fee</code>	Critical	Fixed
RTA-15	<code>is_valid_referral_code()</code> Lacks Length Validation	Minor	Fixed
TRA-12	Missing Account Position Index Sync in Standalone Open Path of <code>match_orders</code> May Cause Hidden Exposure Risk	Medium	Fixed
TRA-14	Missing Collateral-Type Binding in <code>trading</code> Position-Mutation Flows Enables Cross-Vault Drain	Critical	Fixed
TRA-17	Low Collateral Value Prevents Position Closure	Minor	Acknowledged
TRA-18	Insufficient Test Coverage	Minor	Partially Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the [WaterX](#) Smart Contract :

Admin

- **Manage Protocol Versions:** Admin can allow or disallow protocol versions through `allow_version()` and `disallow_version()` .
- **Configure Fees:** Admin can set protocol-wide fees through `set_liquidator_fee_bps()` , `set_insurance_fee_bps()` , and `set_protocol_fee_share()` .
- **Address Management:** Admin can set key protocol addresses through `set_fee_address()` and `set_insurance_address()` .
- **Operator Management:** Admin can manage authorized system keepers through `add_keeper()` and `remove_keeper()` .
- **Global Constraints:** Admin can adjust system-wide constraints through `set_max_orders_per_price()` , `set_oi_cap_bps()` , and `set_price_refresh_threshold_ms()` .
- **Market Lifecycle:** Admin can create and manage trading markets through `create_market()` , `suspend_market()` , and `resume_market()` .
- **Market Configuration:** Admin can update market-specific parameters (leverage, fees, OI caps, funding rates) through `update_market_config()` .
- **Execution Rules:** Admin can manage trading rules and witness requirements through `add_request_rule()` and `add_response_rule()` .
- **Liquidity Pool Control:** Admin can manage the WLP pool through `create_pool()` , `add_token()` , `suspend_pool()` , and `resume_pool()` .
- **Referral Rewards:** Admin can set referral reward rates through the `set_referral_reward_bps()` function.

Keeper

- **Trade Execution:** Keeper can execute pending trading requests through the `execute()` function.
- **Order Matching:** Keeper can facilitate order matching through the `match_orders()` function.
- **Liquidations:** Keeper can trigger liquidations for under-collateralized positions through the `liquidate_request()` .
- **Emergency Actions:** Keeper can perform emergency actions through `open_position_request_by_keeper()` and `close_position_request_by_keeper()` .
- **State Updates:** Keeper can update system state variables through `update_funding_rate()` , `update_borrow_rates()` , and `update_token_value()` .
- **Redeem Settlement:** Keeper (or any automated process) can finalize liquidity withdrawals through `settle_redeem()` after the cooldown period.

User (Trader & Liquidity Provider)

- **Position Management:** User can initiate trading actions through `open_position_request()` , `close_position_request()` , `increase_position_request()` , and `decrease_position_request()` .
- **Collateral Management:** User can manage position collateral through `deposit_collateral_request()` and `withdraw_collateral_request()` .
- **Order Management:** User can manage limit and stop orders through `place_order_request()` and `cancel_order_request()` .
- **Account Management:** User can manage their protocol account through `create_account()` , `deposit_to_account()` , and `withdraw_from_account()` .
- **Permission Delegation:** User can delegate account permissions to other addresses through `delegate_permission()` and `revoke_permission()` .
- **Liquidity Provision:** User can participate as an LP through `mint_wlp()` , `request_redeem()` , and `cancel_redeem()` .

- **Referral System:** User can participate in the referral system through `create_referral_code()` and `bind_referral_code()` .
- **Transaction Completion:** User must complete the transaction lifecycle by destroying hot-potato objects via `destroy_request()` or `destroy_response()` .

4 Findings

FLO-6 `pow()` Function May Abort Unnecessarily

Severity: Informational

Status: Acknowledged

Code Location:

contracts/bucket_framework/sources/float.move#131-144;

contracts/bucket_framework/sources/double.move#146-159

Descriptions:

The `pow()` function is used to compute the power of a floating-point number. It employs an exponentiation-by-squaring algorithm. In each loop iteration, it squares the current base value unconditionally at the end. When the exponent `e` is exactly 1, the final result is already determined and should be `result * cur_base`. However, the function still performs a final squaring operation `mul(cur_base, cur_base)`. This multiplication checks for overflow in the intermediate product. Consequently, a case can occur where `b^1` is representable, but the intermediate product `b^2` overflows, causing the `pow(b, ...)` call to abort unexpectedly. This behavior unnecessarily restricts the valid input range for the function.

Suggestion:

It is recommended to modify the loop condition or logic to skip the final, unnecessary squaring operation when the exponent becomes 1, ensuring the function only performs multiplications that are essential for computing the final result.

Resolution:

The team acknowledged this issue and stated that this is a long-used and proven library, so no modifications will be made to the current version to avoid increasing maintenance complexity.

LIS-3 Potential Performance Issue from Using `vec_map` in `aggregator_map`

Severity: Minor

Status: Acknowledged

Code Location:

`contracts/bucket_oracle/sources/listing.move#21`

Descriptions:

In the `listing.init()` function, the protocol uses `vec_map` for `aggregator_map` when creating the cap.

```
fun init(ctx: &mut TxContext) {  
    let cap = ListingCap {  
        id: object::new(ctx),  
        aggregator_map: vec_map::empty(),  
    };  
    transfer::transfer(cap, ctx.sender());  
}
```

However, there is a known issue in Sui where performance may degrade when the number of elements in a vector exceeds 1000. <https://docs.sui.io/guides/developer/dev-cheat-sheet>

Suggestion:

It is recommended to replace `vec_map` with a more scalable data structure.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LPO-9 Missing Interval Catch-Up in `lp_pool::update_borrow_rates` Causes Permanent Borrow Fee Undercharge

Severity: Major

Status: Fixed

Code Location:

`contracts/waterx_perp/sources/lp_pool.move#416-438`

Descriptions:

`lp_pool::update_borrow_rates` advances the pool-level `cumulative_borrow_rate`. Position borrow fees are settled in `position::calculate_borrow_fee`` from the cumulative delta, and the settled borrow fees are transferred back to the pool during decrease/close flows, forming part of LP revenue.

The current implementation only applies one increment (`cumulative_borrow_rate += borrow_rate`) once the time gate is passed, and then immediately sets `last_borrow_timestamp = now`. It does not accumulate by elapsed intervals. The team confirmed this behavior relies on an off-chain bot to trigger every interval, while also acknowledging that on-chain code cannot guarantee stable triggering or fee correctness. If the off-chain bot misses runs, is delayed, or the market remains inactive for long periods, multiple borrow intervals are permanently skipped and cannot be recovered. This leads to systematic undercharging and direct LP revenue loss. The underestimated borrow fees also propagate to position health and liquidation checks, introducing additional risk-control drift.

Relevant code snippets:

```
// contracts/waterx_perp/sources/lp_pool.move:416-433
if (interval > 0 && now >= tp.last_borrow_timestamp + interval) {
  let borrow_rate = calculate_borrow_rate(...);
  tp.cumulative_borrow_rate = tp.cumulative_borrow_rate + borrow_rate;
```



```
tp.last_borrow_timestamp = now;
}
```

```
// contracts/waterx_perp/sources/position.move:478-480
let rate_delta = cumulative_borrow_rate - position.entry_borrow_index;
((position.collateral_amount as u128) * (rate_delta as u128) / 1_000_000_000 as u64)
```

```
// contracts/waterx_perp/sources/trading.move:1637-1647
public fun update_funding_rate<BASE_TOKEN, LP_TOKEN>(...) {
  global_config.verify_keeper(sender_request.address());
  let market_id = object::id(market);
  market.config.update_funding_rate(market_id, clock.timestamp_ms());
}
```

Suggestion:

It is recommended to enforce on-chain catch-up accrual based on elapsed intervals so that delayed triggering still results in complete historical borrow-fee settlement.

It is recommended to add a dedicated keeper entrypoint for borrow-rate updates, together with multi-runner redundancy and observability alerts to reduce dependence on a single off-chain bot.

It is recommended to apply protective operating controls when update lag exceeds a threshold (for example, controlled degradation or restricted critical operations) to prevent continued LP revenue drift and liquidation-risk distortion.

Resolution:

The team adopted our advice and fixed this issue by replacing one-step interval accrual in

`lp_pool::update_borrow_rates` with elapsed-time proportional accrual, computing `elapsed = now - last_borrow_timestamp` and `accrued_borrow_rate = calculate_borrow_rate_accrual(borrow_rate, elapsed, interval)` before updating `cumulative_borrow_rate`, which can be found at `0112b6e131d7ef13152c0f074a29a8af0dc033f6`.

PRU-1 Simplify Price Freshness Check by Using `get_price_no_older_than()`

Severity: Informational

Status: Acknowledged

Code Location:

contracts/pyth_rule/sources/pyth_rule.move#69

Descriptions:

In the current implementation of `pyth_rule.feed()`, the protocol calls `get_price_unsafe()` to retrieve the Pyth price, and then manually checks whether the price is fresh by comparing the price's publish timestamp with a configured `tolerance_sec`.

```
let pyth_price = pyth::get_price_unsafe(pyth_price_info);
let tolerance_sec =
config.tolerance_sec_map.try_get(&coin_type).destroy_with_default(DEFAULT_TOLERANCE_
let onchain_timestamp = clock.timestamp_ms() / 1000;
let pyth_timestamp = pyth_price.get_timestamp();
let price_opt = if (onchain_timestamp.diff(pyth_timestamp) <= tolerance_sec) {
  let price_val = price::get_price(&pyth_price);
  let expo = price::get_expo(&pyth_price);
  let price_n = i64::get_magnitude_if_positive(&price_val);
  let decimal = i64::get_magnitude_if_negative(&expo);
  let price_m = 10u64.pow(decimal as u8);
  let price_float = float::from_fraction(price_n, price_m);
  option::some(price_float)
} else {
```

However, Pyth already provides a built-in method, `get_price_no_older_than()`, which encapsulates this exact behavior. It returns a price only if it is no older than the specified time threshold, otherwise it fails. By using this method, the protocol can eliminate the need for manual timestamp comparison and simplify the logic, while also relying on Pyth's native freshness guarantees. <https://github.com/pyth-network/pyth->

crosschain/blob/70391d5a4ec0ea5c6b7a4ce5b6736acf305000c4/target_chains/aptos/contracts/souL442

Suggestion:

It is recommended to use `get_price_no_older_than()` instead.

PRU-2 Incomplete Handling of expo Sign in pyth_rule.feed()

Severity: Minor

Status: Acknowledged

Code Location:

contracts/pyth_rule/sources/pyth_rule.move#77

Descriptions:

In the `pyth_rule.feed()` function, the protocol only considers the case where `expo` is negative. However, it does not account for scenarios where `expo` is positive, which should also be handled to ensure correct price interpretation.

```
let price_opt = if (onchain_timestamp.diff(pyth_timestamp) <= tolerance_sec) {  
  let price_val = price::get_price(&pyth_price);  
  let expo = price::get_expo(&pyth_price);  
  let price_n = i64::get_magnitude_if_positive(&price_val);  
  let decimal = i64::get_magnitude_if_negative(&expo);  
  let price_m = 10u64.pow(decimal as u8);  
  let price_float = float::from_fraction(price_n, price_m);  
  option::some(price_float)  
} else {  
  option::none()  
};
```

Suggestion:

It is recommended to update the logic in `pyth_rule.feed()` to handle both negative and positive values of `expo`.

SHE-5 `try_into_float()` Contains Typo in Comment

Severity: Informational

Status: Acknowledged

Code Location:

`contracts/bucket_framework/sources/sheet.move#75`

Descriptions:

The `try_into_float()` function in the `double.move` file is designed to convert a `Double` type value to a `Float` type, with a comment warning about potential precision loss during the conversion. However, the word "WARNING" in the comment is misspelled as "WARING". This is a typographical error in the code documentation. While this does not affect the program's execution, it makes the comment look unprofessional and could cause confusion for developers reading the code, as the intended word is not immediately clear.

Suggestion:

It is recommended to correct the spelling of "WARNING" in the comment to ensure code documentation is accurate and clear.

Resolution:

The team acknowledged this issue and stated that this is a long-used and proven library, so no modifications will be made to the current version to avoid increasing maintenance complexity.

WLP-4 Dependence on Centralized Storage Service for image URL

Severity: Informational

Status: Acknowledged

Code Location:

contracts/wlp/sources/wlp.move#32

Descriptions:

In the `create_wlp_pool()` function, the protocol uses a centralized image URL when creating a new currency. This introduces a dependency on a centralized resource, which may pose risks such as availability issues, lack of immutability, or potential censorship.

```
let (initializer, treasury_cap) = coin_registry.new_currency<WLP>(
  decimals(),
  b"WLP".to_string(),
  b"WaterX LP Token".to_string(),
  b"Liquidity Provider token for WaterX Perp DEX".to_string(),
  b"https://token-metadata.bridge.xyz/images/usd_sui.png".to_string(),
  ctx,
);
```

Suggestion:

It is recommended to use decentralized storage solutions such as IPFS or Arweave.

GCO-13 Missing Events in Admin Functions

Severity: Minor

Status: Acknowledged

Code Location:

contracts/waterx_perp/sources/global_config.move#113-205

Descriptions:

The `waterx_perp` protocol contains 30 administrative functions gated by `AdminCap` across 5 modules that manage critical protocol parameters — including keeper addresses, fee recipients, insurance fund addresses, market/pool lifecycle, and oracle rule configuration.

In `global_config.move`, 12 functions (`allow_version()`, `disallow_version()`, `set_liquidator_fee_bps()`, `set_insurance_fee_bps()`, `set_max_orders_per_price()`, `set_oi_cap_bps()`, `set_price_refresh_threshold_ms()`, `add_keeper()`, `remove_keeper()`, `set_protocol_fee_share()`, `set_fee_address()`, `set_insurance_address()`) modify protocol-wide settings. In `market_config.move`, 7 functions (`update_market_config()`, `suspend_market()`, `resume_market()`, `add_request_rule()`, `remove_request_rule()`, `add_response_rule()`, `remove_response_rule()`) control per-market configuration and oracle rule checklists. In `trading.move`, `create_market()` deploys new trading markets. In `lp_pool.move`, 6 functions (`create_pool()`, `allow_version()`, `disallow_version()`, `add_token()`, `suspend_pool()`, `resume_pool()`) manage the WLP liquidity pool. In `referral_table.move`, 4 functions (`add_version()`, `remove_version()`, `allow_version()`, `disallow_version()`) manage the referral system versioning.

However, none of these 30 functions emit any events when they are called and successfully executed. Events are essential for off-chain applications and users to track and verify changes to the protocol's state in a transparent and reliable manner. Without events, security-sensitive operations such as changing the fee or insurance fund recipient address,

adding or removing keepers, and suspending markets or the LP pool are invisible to off-chain monitoring systems and indexers, leaving no on-chain audit trail for protocol governance changes.

Suggestion:

It is recommended to emit events for all administrative functions.

Resolution:

The team acknowledged this issue and stated that future implementations will be added based on ongoing requirements, so no modifications will be made to the current version.

KBV-19 State Bloat Caused by Dynamic Fields Leaking in `drop` of `KeyedBigVector`

Severity: Minor

Status: Acknowledged

Code Location:

`contracts/waterx_perp/sources/keyed_big_vector.move#293-303`

Descriptions:

The `waterx_perp::keyed_big_vector` module establishes a custom persistent data container structure designated as `KeyedBigVector`. This specific smart contract integrates a public `drop` function tailored to fully dismantle and safely burn initialized state segments mapping to instances of this object type.

The underlying logic vulnerability manifests internally inside the `drop` function call execution phase. The code decomposes variables but abruptly ends the hierarchy via the `id.delete()` instruction. In relation to Sui Move architecture fundamentals, deliberately destroying the native root `UID` identity strictly ignores downstream dependencies and omits cascade-deleting related configurations like attached dynamic fields. Functionally, `KeyedBigVector` deposits heavy storage amounts allocating elements onto instantiated `Slice` modules, structurally grafting them identically as sub-dynamic structures linked with the deleted `UID`. The `drop` function completely bypasses any retrieval operations necessary to properly decouple, consume, and drop these sub-elements dynamically correctly.

Executing this flow invokes permanent memory locking behaviors and serious resource stranding outcomes. With the parent framework's `KeyedBigVector` entity explicitly revoked from the operational sphere, every previously attached underlying `Slice` artifact descends irreversibly into ghost states completely decoupled from reference registries. Operating exclusively as un-referenced nodes stranded continuously into general block persistence parameters prevents end-users or systems architectures uniformly from collecting their

applicable network storage rebates. This permanent limitation produces direct on-chain scaling friction manifesting heavily through cumulative state bloat side-effects.

Suggestion:

It is recommended to introduce an interactive cleanup routine validating complete structural dismantling prior to utilizing object deletion parameters. Manually and methodically traverse, detach and effectively destroy every corresponding dynamically mapped resource structures encompassing all indexically assigned `Slice` occurrences across the parent object. Explicit iteration to consume dynamically allocated children prevents orphaned entries on the shared blockchain environment.

Resolution:

The team acknowledged this issue and state that "We will not have operations to delete `KeyedBigVector` objects, so the scenario described in the report is unlikely to occur", so no modifications will be made to the current version.

LPO-20 Inconsistent Usage of Basis Point Scale Constant May Lead to Maintainability Issues

Severity: Informational

Status: Acknowledged

Code Location:

contracts/waterx_perp/sources/lp_pool.move

Descriptions:

Functions such as `check_reserve_valid` within the `lp_pool` module are responsible for verifying if the reserved amount in the token pool strictly remains within the maximum allowed ratio. Meanwhile, various functions in the `trading` and `global_config` modules are used to calculate, apply, or validate protocol fee shares and margin rules. All these logic instances entail mathematical calculations parameterized by a standard Basis Point (BPS) factor.

In the logic implementations within `lp_pool`, the calculations appropriately utilize the encapsulated `math::bp_scale()` function from the `math` module to yield the BPS value as a denominator. Conversely, inside other core business logic paths, including the fee computation in the `trading` module and checks inside the `global_config` module, the magic number `10000` is hardcoded directly. This indicates that two entirely discrete mechanisms are being adopted simultaneously across the codebase to execute the identical constant reference pattern.

Although the return value from `math::bp_scale()` evaluates to exactly `10000`—preventing any immediate calculation errors in the current version—this coding inconsistency and the absence of centralized constant management undermine the overall project's maintainability. Over extended phases of version iterations and feature updates, failing to uniformly bind the same semantic constant to a centralized function significantly elevates the risk of incomplete future code refactoring, raising the propensity for unintended logic

bugs and subsequent incorrect financial calculations due to simple copy-paste or typographical errors.

Suggestion:

It is recommended to replace all instances of the hardcoded `10000` value used for basis point configurations and mathematical calculations throughout the codebase with direct calls to the `math::bp_scale()` function. This standardizes constant referencing, enhances code predictability, and ultimately mitigates latent maintainability tracking risks.

Resolution:

The team acknowledged this issue and stated that "the Basis Point is fixed at 10000; although this might lead to potential misleading in future upgrades, it is currently not in the repair plan based on their evaluation", so no modifications will be made to the current version.

MCO-10 Missing Multi-Interval Funding Accrual in `market_config::update_funding_rate`

Severity: Major

Status: Fixed

Code Location:

`contracts/waterx_perp/sources/market_config.move#335-367`

Descriptions:

`market_config::update_funding_rate` is responsible for advancing the market-level cumulative funding index, and it is invoked by open/increase/decrease/close/liquidation/order-matching flows and by the keeper funding update entry. The current logic only checks whether `now_ms >= last_funding_timestamp + interval`, applies a single funding increment, and then sets `last_funding_timestamp` directly to `now_ms`. As a result, if multiple intervals have elapsed before the next update call, the contract does not backfill missed intervals and accrues only one interval of funding. Because position-level funding settlement is derived from the cumulative funding index delta, this under-accrual directly reduces payable/receivable funding on positions. A practical path is: during keeper inactivity or sparse market interactions, a trader on the paying side waits through multiple intervals and then closes/reduces when interaction resumes. The trader is charged for only one interval instead of all elapsed intervals, allowing funding-cost avoidance while reducing expected funding transfers to counterparties/LPs and weakening imbalance correction.

Relevant Code:

`contracts/waterx_perp/sources/market_config.move:335-367`

```
public(package) fun update_funding_rate<BASE_TOKEN>(  
  market: &mut MarketConfig<BASE_TOKEN>,  
  market_id: ID,  
  now_ms: u64,
```

```

): bool {
  let interval = market.funding_interval_ms();
  if (now_ms < market.last_funding_timestamp() + interval) return false;
  let long_oi = market.long_oi();
  let short_oi = market.short_oi();
  let basic_rate = market.basic_funding_rate();
  let (sign, rate_float) = calculate_funding_rate(long_oi, short_oi, basic_rate);
  let rate_double = rate_float.into_double();
  let current_sign = market.cumulative_funding_sign();
  let current_index = market.cumulative_funding_index();

  let (new_sign, new_index) = if (sign == current_sign) {
    (sign, current_index.add(rate_double))
  } else {
    if (rate_double.gte(current_index)) {
      (sign, rate_double.sub(current_index))
    } else {
      (current_sign, current_index.sub(rate_double))
    }
  };

  market.update_funding_state(new_sign, new_index, now_ms);
  events::emit_funding_rate_updated(
    market_id, rate_float, new_index, long_oi, short_oi, now_ms,
  );
  true
}

```

contracts/waterx_perp/sources/trading.move:790-807

```

pool.update_borrow_rates(clock);
market.config.update_funding_rate(market_id, now);
...
pos.update_fees(price, collateral_price, cumulative_borrow, funding_sign,
funding_index);

```

contracts/waterx_perp/sources/position.move:490-509

```

let (delta_sign, delta_index) = if (cumulative_funding_sign == position.entry_funding_sign)
{
  if (cumulative_funding_index.gte(position.entry_funding_index)) {
    (cumulative_funding_sign,
cumulative_funding_index.sub(position.entry_funding_index))
  } else {
    (!cumulative_funding_sign,
position.entry_funding_index.sub(cumulative_funding_index))
  }
} else {
  (cumulative_funding_sign,
cumulative_funding_index.add(position.entry_funding_index))
};

let period_funding = (
  (size_in_collateral as u256) * delta_index.to_scaled_val()
  / double::precision()
  as u64
);

```

Suggestion:

It is recommended to accrue funding based on the number of elapsed intervals, keep cumulative index growth consistent with elapsed time, advance `last_funding_timestamp` to an interval-aligned settlement boundary, and add regression tests for “first update after multi-interval gap” and “close/reduce immediately after a long inactivity gap” to ensure funding cannot be bypassed via inactivity.

Resolution:

The project now accrues funding linearly with elapsed time, backfilling cumulative funding by the number of fully elapsed funding intervals, while carrying forward sub-interval remainder time to future updates. This addresses the prior under-accrual risk during keeper inactivity or sparse market activity windows. Fix commit: 0112b6e131d7ef13152c0f074a29a8af0dc033f6

POS-21 Incorrect Borrow Fee Calculation Based on Collateral Amount in `calculate_borrow_fee`

Severity: Critical

Status: Fixed

Code Location:

`contracts/waterx_perp/sources/position.move#474-481`

Descriptions:

The `calculate_borrow_fee` function in `position.move` is responsible for calculating the borrow fee a user needs to pay for their open position over a period of time.

In the current implementation, the borrow fee is calculated by multiplying the `rate_delta` (the difference in the cumulative borrow rate) by the user's collateral amount (`position.collateral_amount`). This calculation logic is fundamentally flawed from an economic modeling perspective. Normally, in perpetual contracts and lending protocols, the borrow fee should be based on the actual amount of funds the user "borrows" from the protocol—which is typically the nominal value of the position minus the user's provided collateral, or simply the total nominal position size.

```
fun calculate_borrow_fee(  
  position: &Position,  
  cumulative_borrow_rate: u64,  
) : u64 {  
  if (cumulative_borrow_rate <= position.entry_borrow_index) return 0;  
  let rate_delta = cumulative_borrow_rate - position.entry_borrow_index;  
  ((position.collateral_amount as u128) * (rate_delta as u128) / 1_000_000_000 as u64)  
}
```

By using the `collateral_amount` as the multiplier base, it leads to a highly unreasonable scenario: users with lower leverage (who provide more collateral and borrow less from the protocol) are penalized by having to pay a higher borrow fee. Conversely, users with higher

leverage (who provide less collateral and implicitly borrow more) pay significantly less in borrow fees. This creates an incorrect economic incentive design that negatively impacts low-risk users, unfairly encourages high-leverage trading, and presents severe fairness and potential insolvency risks to the protocol.

Suggestion:

It is recommended to revise the borrow fee calculation mechanism within the protocol. The fee should be calculated based on the actual borrowed amount (e.g., nominal position size minus collateral amount) or the total position size, rather than relying solely on the user's collateral amount.

Resolution:

The team adopted our advice and fixed this issue by changing the borrow fee calculation base from the user-provided collateral amount to the actual borrowed amount (nominal value minus collateral). By introducing the `calculate_reserve_amount` function, the protocol now calculates fees based on the actual borrowed principal—derived by subtracting the user's collateral from the position's value—ensuring that fees are aligned with real risk exposure. These changes can be found in commit

43f49f9fb95d8eb8a8a0b613f3bcf89d00f4b9b9.

RTA-15 `is_valid_referral_code()` Lacks Length Validation

Severity: Minor

Status: Fixed

Code Location:

`contracts/waterx_perp/sources/referral_table.move#148-156`

Descriptions:

The `is_valid_referral_code()` function in the referral system is responsible for checking if a referral code is valid. It verifies that each character in the code is either a digit (0-9) or a lowercase letter (a-z). However, the function does not check the length of the referral code. This means that extremely long codes, very short codes, or even empty strings can pass the validation. An empty string or a code with only one character might be considered valid, which could be unintended. This lack of length validation could lead to the acceptance of invalid or malformed referral codes, potentially causing issues in the referral tracking logic.

Suggestion:

It is recommended to add length validation to the `is_valid_referral_code()` function to ensure that referral codes meet a specific minimum and maximum length requirement before being considered valid.

Resolution:

The team adopted our advice and fixed this issue by adding a minimum length limit for invitation codes, which can be found at `cbffa8e044b00dffaafe53569fe9e72ca8abe36a`.

TRA-12 Missing Account Position Index Sync in Standalone Open Path of `match_orders` May Cause Hidden Exposure Risk

Severity: Medium

Status: Fixed

Code Location:

`contracts/waterx_perp/sources/trading.move#1601-1612`

Descriptions:

The `waterx_perp::trading` module is designed to maintain both market-side position storage (`market.positions`) and account-side position indexing (`UserAccount.positions`) for account-level position discovery and management. In the normal open flow, `execute_open_position` writes to `market.positions` and then synchronizes the account index via `user_account::add_position` (`contracts/waterx_perp/sources/trading.move:617-618`).

However, in the standalone order-match open branch of `match_orders` (when `linked_position_id == none`), the code creates and inserts a position only into market storage (`contracts/waterx_perp/sources/trading.move:1601-1612`) without updating the account index. In the same function, the close path does call `user_account::remove_position` (`contracts/waterx_perp/sources/trading.move:1575`), which indicates that account index synchronization is expected behavior.

This creates an on-chain state divergence: a position exists in market storage but may be missing from account views. Frontends, risk engines, bots, and operational tooling that rely on `UserAccount.positions` can miss live exposure, causing delayed close/reduce actions, underestimated risk, and potentially liquidation-driven losses under volatile conditions. This issue usually does not directly enable unauthorized asset transfer, but it materially degrades position visibility and risk-control reliability.

Relevant code evidence:

```
// contracts/waterx_perp/sources/trading.move:1601-1612
let position_id = market.config.increment_next_position_id();
let pos = position::create_position<C_TOKEN>(
  position_id, user, market_id,
  is_long, size, market.config.size_decimal(),
  collateral, global_config.balance_mut<C_TOKEN>(), collateral_decimal,
  price, cumulative_borrow, funding_sign, funding_idx,
  trading_fee, clock, ctx,
);
pool.increase_reserve(collateral_token, fill_reserve_amount);
market.config.adjust_oi(is_long, true, size);
market.positions.push_back(position_id, pos); // missing user_account::add_position(...)
filled_position_id = position_id;
```

```
// contracts/waterx_perp/sources/trading.move:617-618 (normal open path)
market.positions.push_back(position_id, pos);
user_account::add_position(account_registry, account_id, market_id, position_id);
```

```
// contracts/waterx_perp/sources/trading.move:1575 (close branch in same function)
user_account::remove_position(account_registry, user.to_id(), market_id, pos_id);
```

Suggestion:

It is recommended to enforce a unified account-index synchronization rule across all position-creation paths so that `market.positions` and `UserAccount.positions` remain atomically consistent. It is recommended to add state-consistency invariants and regression tests covering market open, standalone order-match open, linked order increase/decrease, and liquidation/force-close paths. It is recommended to make off-chain indexing and monitoring cross-check `OrderFilled` events against market-side position storage until the fix is fully deployed.

Resolution:

The team adopted our advice and fixed this issue by adding `user_account::add_position(account_registry, user.to_id(), market_id, position_id)` in the

standalone open path of match_orders to synchronize account position indexing with market-side position storage. This resolves the state divergence where a position exists in market storage but is missing from account views. The fix can be found at commit 43f49f9fb95d8eb8a8a0b613f3bcf89d00f4b9b9.

TRA-14 Missing Collateral-Type Binding in trading Position-Mutation Flows Enables Cross-Vault Drain

Severity: Critical

Status: Fixed

Code Location:

contracts/waterx_perp/sources/trading.move#1208

Descriptions:

In `waterx_perp::trading`, existing-position mutation flows (`close/decrease/withdraw/deposit/liquidate`) select `GlobalVault<C_TOKEN>` and run fee/collateral settlement based on the runtime generic `C_TOKEN`. However, these paths do not enforce that `position.collateral_token` matches `C_TOKEN`. A collateral-type check exists only in a specific linked-order branch, leaving core position flows without the same invariant. Root Cause: The real collateral type recorded in the position is decoupled from the runtime type argument used during execution. Request constructors (for example `close_position_request` and `decrease_position_request`) bind position/account permissions but do not bind or verify collateral type. During execution, the code directly uses `C_TOKEN` to fetch borrow indexes, collateral pricing, and `GlobalVault<C_TOKEN>`, then moves balances through functions that split by numeric `collateral_amount` only. Since `position::remove_position` and `position::withdraw_collateral` do not validate token-type consistency, an authorized caller can force value extraction from the wrong vault type.

Impact:

1. A victim opens a position with a high-value collateral asset (for example USDC), funding `GlobalVault<USDC>`.
2. The attacker opens their own position with a low-value collateral asset (for example CHEAP), obtaining a valid position with a large numeric `collateral_amount`.

3. The attacker closes their CHEAP-collateral position but intentionally calls the flow with `C_TOKEN=USDC` .
4. Ownership checks still pass (the attacker is operating on their own position), while execution reads and debits `GlobalVault<USDC>` because the flow trusts runtime `C_TOKEN` .
5. The attacker receives USDC extracted from the USDC vault, turning a cheap-asset position into high-value asset withdrawal.

Impact: Direct theft from other users/protocol collateral pools, cross-vault insolvency risk, and corrupted fee/health accounting that can further distort liquidation and risk controls.

Suggestion:

It is recommended to enforce a strict invariant across all existing-position entry and execution paths: the position's recorded `collateral_token` must match the execution asset type, otherwise revert immediately.

It is recommended to make position-collateral type position-derived for all mutation flows, instead of allowing external type parameters to determine vault read/write targets.

Resolution:

The project team has remediated this issue in commit

`cbffa8e044b00dffaafe53569fe9e72ca8abe36a` . The fix enforces collateral-type consistency across existing-position request/execute paths by binding operations to the position's recorded collateral type and rejecting mismatched `C_TOKEN` usage with `EInvalidCollateralType` (208). In addition, position-level vault mutation functions now assert token-type consistency before any split/join movement, which removes the cross-vault type-confusion drain primitive.

TRA-17 Low Collateral Value Prevents Position Closure

Severity: Minor

Status: Acknowledged

Code Location:

contracts/waterx_perp/sources/trading.move#580-590

Descriptions:

If the collateral value is too low to close the position, and assuming the collateral value is less than `total_fee`, the `split` will cause an `abort` when attempting to close the position, preventing the position from being closed.

```
...
let total_fee = close_fee + borrow_fee + open_fee.min(collateral.value());
if (total_fee > 0) {
  let mut fee_balance = collateral.split(total_fee);
  let protocol_amount = ((fee_balance.value() as u128) * (protocol_share_bps as
u128) / 10000 as u64); // @audit inf 10000 should be take to BPS_SCALE
  if (protocol_amount > 0) {
    let protocol_fee = fee_balance.split(protocol_amount);
    transfer::public_transfer(protocol_fee.into_coin(ctx), protocol_fee_addr);
    events::emit_protocol_fee_collected(protocol_fee_addr, protocol_amount,
market_id, now);
  };
  pool.put_collateral(fee_balance, collateral_price);
};
...
```

Suggestion:

If the value of the collateral is too low, we recommend using it to cover all fees and then closing the position immediately.

Resolution:

Developer's response: When this situation arises, the position should be liquidated; therefore, we will continue to improve the stability of Keeper and Liquidator.

TRA-18 Insufficient Test Coverage

Severity: Minor

Status: Partially Fixed

Code Location:

contracts/waterx_perp/sources/trading.move

Descriptions:

The current unit test suite does not sufficiently cover several critical business-logic paths involving LP liquidity and trader settlement. Although standard trading and basic LP flows are tested, important edge cases remain only partially covered, including scenarios such as low-liquidity profit settlement, LP redemption interacting with open positions, liquidation near collateral and fee boundaries, collateral modification near liquidation thresholds, key rejection branches such as slippage or insufficient collateral, etc.

Suggestion:

It is recommended to add enough tests to ensure that every part of the design matches the expected design.

Resolution:

The team adopted our advice and partially fixed this issue by implementing the recommended fix. However, insufficient unit test coverage still exists in the current code version. The team acknowledged this issue and stated that they are still continuously updating and supplementing the unit tests.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

